

Trabalho Prático 1: Sistema de Busca Ortogonal com KD-Tree

Isadora Cunha Pimentel
isadorac@ufmg.br

Pedro Bernardo Goulart Parreira
pedrobgoulart@ufmg.br

Maio de 2026

Abstract

Este relatório descreve o desenvolvimento de um sistema para consulta de bares participantes do Comida Di Buteco 2026 em Belo Horizonte. O sistema implementa busca ortogonal utilizando uma KD-Tree, permitindo ao usuário informar um endereço e um raio para localizar butecos, com suporte a geometrias retangular e circular. A interface web foi desenvolvida com Dash e dash-leaflet, e encontra-se hospedada no Render. O trabalho aborda desde a coleta e tratamento dos dados até a implementação da estrutura de dados e a construção da interface interativa.

1 Introdução

O objetivo do trabalho foi aplicar conceitos de geometria computacional na construção de um sistema de busca espacial. Utilizamos uma base com 109 butecos fornecida pelo professor e desenvolvemos uma aplicação em Python com as bibliotecas Dash, dash-leaflet e geopy. O sistema permite que o usuário digite um endereço, informe um raio de busca em quilômetros e visualize no mapa todos os butecos localizados dentro da região definida, podendo alternar entre busca retangular e circular. Além da visualização no mapa, os resultados são exibidos em uma tabela interativa ordenada por distância, com informações de nome, endereço e distância até o centro da busca. O projeto foi desenvolvido em dupla, versionado com Git em repositório público no GitHub, e encontra-se hospedado no Render.

2 Estrutura de Dados: KD-Tree

A KD-Tree (k-dimensional tree) é uma estrutura de dados utilizada para organizar pontos em um espaço k-dimensional. Neste trabalho, trabalhamos com $k=2$, onde as duas dimensões são latitude e longitude. A implementação foi feita inteiramente em Python no arquivo KDTree.py.

Cada nó da árvore contém quatro elementos: um ponto representado por uma tupla (lat, lon), o eixo de divisão (0 para latitude, 1 para longitude), e as referências para as subárvores esquerda e direita. A escolha do eixo alterna a cada nível de profundidade: na raiz dividimos por latitude, nos filhos por longitude, nos netos novamente por latitude, e assim sucessivamente.

A construção da árvore é feita pela função `construir_KDTree`. O algoritmo recebe uma lista de pontos e a profundidade atual. A cada chamada recursiva, os pontos são ordenados pelo eixo correspondente. O ponto mediano torna-se a raiz da subárvore atual. Os pontos à esquerda da mediana são usados para construir recursivamente a subárvore esquerda, e os pontos à direita para a subárvore direita. A complexidade de construção é $O(n \log n)$.

A busca ortogonal é implementada pela função `pontos_relevantes` para geometria retangular. Esta função recebe a raiz da árvore, os limites do retângulo e a profundidade atual. Para cada nó visitado, verifica-se se o ponto está dentro do retângulo e, usando o eixo do nó, decide-se quais subárvores precisam ser exploradas, podendo galhos que não intersectam a região de busca.

3 Implementação

Os dados de entrada são dois arquivos. O arquivo `coordinates.txt` contém pares de coordenadas geográficas no formato "latitude,longitude". O arquivo `buteos.bh.csv` contém informações cadastrais dos bares com as colunas "name" (nome) e "address" (endereço). Um desafio importante foi que o arquivo de coordenadas não contém nenhum identificador que permita associar cada coordenada ao buteco correspondente no CSV. A solução adotada foi associar os dados sequencialmente: o primeiro ponto do arquivo

de coordenadas corresponde ao primeiro buteco do CSV, o segundo ponto ao segundo buteco, e assim por diante.

A geocodificação dos endereços é feita com a biblioteca `geopy`, que consulta a API do Nominatim (OpenStreetMap). A função `descobre_coordenadas` recebe um endereço digitado pelo usuário, adiciona "Belo Horizonte, MG, Brasil" para contextualizar a busca, e retorna as coordenadas correspondentes.

A interface web foi construída com Dash. O layout é composto por um título, uma barra de busca com campo de endereço e botão de pesquisa, um campo para informar o raio em quilômetros, um seletor de geometria (retângulo ou círculo), um botão para limpar a seleção, o mapa (componente `dash-leaflet`), e uma tabela para exibir os resultados.

3.1 Busca Retangular e Circular

A atividade extra proposta no enunciado foi implementada: o usuário pode alternar entre busca retangular e circular usando um seletor no painel de controle.

A busca retangular utiliza a função `pontos_relevantes` que percorre a KD-Tree aplicando poda baseada nos limites do retângulo. A busca circular implementa uma estratégia de aproximação: primeiro calcula-se um bounding box retangular ao redor do círculo para poda na KD-Tree, e então calcula-se a distância geodésica exata usando a fórmula de Haversine (via `geodesic` do `geopy`) para cada ponto candidato, mantendo apenas aqueles dentro do raio informado.

No mapa, a região de busca é visualizada através de elementos distintos: para o retângulo, utiliza-se um polígono com opacidade e borda destacada; para o círculo, utiliza-se um marcador circular com raio proporcional ao valor informado.

3.2 Ordenação e Visualização dos Resultados

Os pontos filtrados são ordenados por distância crescente em relação ao centro da busca usando a função `ordena_butecos`. A tabela exibe nome, endereço e distância formatada com duas casas decimais. Os pop-ups no mapa também apresentam as mesmas informações, garantindo consistência entre as visualizações.

4 Dificuldades Encontradas

A associação entre coordenadas e butecos foi um desafio inicial. A solução foi assumir a ordem sequencial dos arquivos, que se mostrou correta para os dados fornecidos.

O gerenciamento de callbacks no Dash apresentou conflitos, especialmente com múltiplos outputs. A solução foi combinar outputs relacionados em um único callback e utilizar `allow_duplicate=True` quando necessário.

A implementação da busca circular exigiu cuidados adicionais, pois não se trata de uma busca ortogonal nativa na KD-Tree. A estratégia adotada foi utilizar um bounding box retangular para poda inicial, seguido de verificação exata de distância.

A geocodificação via Nominatim impôs limitações de taxa e alguns endereços retornavam resultados imprecisos. Foram implementados tratamentos de erro e mensagens informativas.

A estilização dos componentes no Dash foi trabalhosa devido à estrutura HTML complexa gerada. Utilizamos a ferramenta de inspeção do navegador para identificar classes e direcionar os estilos via pasta `assets/`.

A performance da KD-Tree foi otimizada construindo a árvore uma única vez durante a inicialização da aplicação.

O trabalho colaborativo com Git gerou conflitos que exigiram comunicação frequente e resolução cuidadosa de merges.

5 Deploy no Render

A aplicação foi hospedada no Render, uma plataforma de nuvem que suporta aplicações Python. Para o deploy, foram preparados os seguintes arquivos:

- **requirements.txt:** contendo as dependências necessárias (`dash`, `dash-leaflet`, `dash-extensions`, `geopy`, `unicorn`)
- **.python-version:** especificando a versão 3.11 do Python para compatibilidade

- **render.yaml**: arquivo de configuração do serviço web

O comando de inicialização utilizado foi `gunicorn main:app`, que executa o servidor WSGI. O deploy foi realizado conectando o repositório GitHub ao Render, que automaticamente detecta a aplicação Python e executa o build.

6 Conclusão

O trabalho permitiu aplicar na prática os conceitos de KD-Tree e busca espacial estudados em sala de aula. O sistema resultante é funcional, interativo e oferece suporte tanto a buscas ortogonais retangulares quanto a buscas circulares. O mapa responde dinamicamente às entradas do usuário, a região de busca se ajusta automaticamente, e os pontos são filtrados corretamente. A integração entre a barra de busca, o mapa e a tabela proporciona uma experiência de usuário fluida.

Como melhorias futuras, seria interessante permitir que o usuário arraste a região de busca diretamente no mapa, implementar um modo responsivo mais robusto para dispositivos móveis, e adicionar a funcionalidade de exibir a rota até o buteco selecionado.